

# Introduction to Supervised Learning

## From Features to Targets

Mehmet Mercangöz

AI and AIS Summer School July/Aug 2026

## ① Foundations

- supervised learning, labels, features, regression, classification

## ② Learning as optimisation

- loss functions, least squares, polynomial regression

## ③ Generalisation and evaluation

- train/validation/test split, overfitting, metrics

## ④ Classification models

- logistic regression, decision thresholds, confusion matrix

## ⑤ Practical modelling choices

- data preparation, model selection

# What Is Supervised Learning?

Supervised learning uses examples where the desired answer is already known.

Each training example contains:

input features  $\mathbf{x}_i$   $\longrightarrow$  target or label  $y_i$ .

The goal is to learn a prediction rule

$$\hat{y} = f(\mathbf{x})$$

that gives useful predictions for **new, unseen** inputs.

## Core idea

The model should not merely remember the training data. It should **generalise**.

# Supervised, Unsupervised, and Reinforcement Learning

| Setting                | Data                      | Typical question                           |
|------------------------|---------------------------|--------------------------------------------|
| Supervised learning    | Inputs with known targets | Can we predict the target for a new input? |
| Unsupervised learning  | Inputs without targets    | Are there patterns, groups, or structure?  |
| Reinforcement learning | Actions, states, rewards  | Which actions should be taken over time?   |

This lecture focuses on **supervised learning**.

# Regression and Classification

Supervised learning problems usually fall into two main types.

## Regression

Predict a numerical value.

Examples:

- house price;
- energy demand;
- product quality;
- reactor conversion.

## Classification

Predict a category or class.

Examples:

- spam or not spam;
- faulty or healthy;
- image class;
- high-risk or low-risk.

# The Supervised Learning Dataset

Suppose we have  $n$  examples and  $p$  features.

$$X = \begin{bmatrix} - & - & - & \mathbf{x}_1^\top & - & - & - \\ - & - & - & \mathbf{x}_2^\top & - & - & - \\ & & & \vdots & & & \\ - & - & - & \mathbf{x}_n^\top & - & - & - \end{bmatrix} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}.$$

- Each **row** is one observation.
- Each **column** is one feature.
- The target  $y_i$  is what we want to predict.

# A Small Example Dataset

| Observation | Temperature | Pressure | Quality |
|-------------|-------------|----------|---------|
| 1           | 80          | 2.1      | 91.2    |
| 2           | 83          | 2.4      | 92.8    |
| 3           | 77          | 1.9      | 89.7    |
| 4           | 85          | 2.5      | 94.1    |

Here, a possible supervised learning task is:

$(\text{temperature}, \text{pressure}) \mapsto \text{quality}.$

## Question

Is this a regression problem or a classification problem?

# The Supervised Learning Workflow

- 1 Define the prediction problem.
- 2 Identify the inputs  $\mathbf{x}$  and target  $y$ .
- 3 Prepare and inspect the data.
- 4 Split the data into training, validation, and test sets.
- 5 Choose a model class.
- 6 Train the model by minimising a loss function.
- 7 Evaluate the trained model on unseen data.
- 8 Inspect errors and improve the model if needed.

## Important

The algorithm is only one part of the workflow. Problem formulation and data quality are often just as important.



# What Does It Mean for a Model to Learn?

A model usually contains adjustable parameters.

$$\hat{y}_i = f(\mathbf{x}_i; \boldsymbol{\theta}).$$

Training means choosing parameters  $\boldsymbol{\theta}$  so that predictions match the observed targets as well as possible.

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} L(\boldsymbol{\theta}).$$

- $L(\boldsymbol{\theta})$  is the **loss function**.
- It measures prediction error on the training data.
- Supervised learning is therefore closely connected to optimisation.

# Identify: Problem Type, Targets, Features

## Application

## Regression or Classification?

Predict tomorrow's electricity demand using weather and calendar data

Determine whether an incoming email is spam

Estimate product purity from process measurements

Determine whether a machine is healthy or faulty

# Linear Regression

Linear regression predicts a numerical target using a weighted sum of features.

$$\hat{y}_i = w_0 + w_1x_{i1} + w_2x_{i2} + \cdots + w_px_{ip}.$$

Using a bias column of ones, this can be written compactly as

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}.$$

- $\mathbf{X}$  contains the input data.
- $\mathbf{w}$  contains the coefficients to be estimated.
- $\hat{\mathbf{y}}$  contains the model predictions.

# Residuals and Least-Squares Loss

For each training example, the residual is

$$e_i = y_i - \hat{y}_i.$$

For all examples together:

$$\mathbf{e} = \mathbf{y} - X\mathbf{w}.$$

The least-squares objective is

$$J(\mathbf{w}) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \|\mathbf{y} - X\mathbf{w}\|_2^2.$$

Choose the coefficients that make the total squared prediction error as small as possible.

# Least-Squares Solution

The least-squares problem is

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \|\mathbf{y} - X\mathbf{w}\|_2^2.$$

Expanding the objective:

$$J(\mathbf{w}) = (\mathbf{y} - X\mathbf{w})^\top (\mathbf{y} - X\mathbf{w}).$$

Taking the gradient and setting it equal to zero gives

$$\nabla_{\mathbf{w}} J = 2X^\top (X\mathbf{w} - \mathbf{y}) = \mathbf{0}.$$

Therefore,

$$X^\top X\mathbf{w} = X^\top \mathbf{y}.$$

These are called the **normal equations**.

# Closed-Form Linear Regression Estimate

If  $X^\top X$  is invertible, the least-squares estimate is

$$\hat{\mathbf{w}} = (X^\top X)^{-1} X^\top \mathbf{y}$$

This is a direct solution of an optimisation problem.

When is this formula safe to use?

It requires  $X^\top X$  to be invertible and reasonably well-conditioned.

If this is not the case, we often use the pseudoinverse:

$$\hat{\mathbf{w}} = X^+ \mathbf{y}.$$

# Least Squares: Geometric Interpretation

The model predictions have the form

$$\hat{\mathbf{y}} = X\hat{\mathbf{w}}.$$

This means  $\hat{\mathbf{y}}$  must lie in the column space of  $X$ .

Least squares chooses the point in that space that is closest to  $\mathbf{y}$ .

$$\mathbf{y} = \hat{\mathbf{y}} + \mathbf{e}, \quad X^T \mathbf{e} = \mathbf{0}.$$

## Key idea

The residual vector is orthogonal to every column of the design matrix.

# Linear Regression: Parameters and Hyperparameters

In linear regression, the coefficients are **parameters**.

$$\mathbf{w} = [w_0 \quad w_1 \quad \cdots \quad w_p]^\top.$$

They are learned from the data by solving an optimisation problem.

A **hyperparameter** is chosen by the modeller before training.

Examples:

- the degree of a polynomial regression model;
- the number of neighbours in  $k$ -nearest neighbours;
- the maximum depth of a decision tree;
- the strength of regularisation.



# Polynomial Regression

Suppose the relationship between  $x$  and  $y$  is curved.

A polynomial regression model may use

$$\hat{y} = a_0 + a_1x + a_2x^2 + a_3x^3.$$

This is nonlinear as a function of  $x$ , but it is still **linear in the unknown coefficients**:

$$\mathbf{a} = [a_0 \quad a_1 \quad a_2 \quad a_3]^\top.$$

Therefore, estimating the coefficients is still a linear least-squares problem.

# Polynomial Regression as Linear Least Squares

For training data  $(x_i, y_i)$ , define a new design matrix:

$$\Phi = \begin{bmatrix} 1 & x_1 & x_1^2 & x_1^3 \\ 1 & x_2 & x_2^2 & x_2^3 \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 & x_n^3 \end{bmatrix}.$$

The model becomes

$$\hat{\mathbf{y}} = \Phi \mathbf{a}.$$

The coefficient estimate is

$$\hat{\mathbf{a}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{y}$$

when  $\Phi^\top \Phi$  is invertible.

# Polynomial Regression: Important Lesson

Nonlinear features do not necessarily mean nonlinear optimisation.

A model can be nonlinear in the input  $x$ , but linear in the unknown coefficients.

Examples of linear-in-parameters models:

$$\hat{y} = a_0 + a_1x + a_2x^2,$$

$$\hat{y} = a_0 + a_1 \sin(x) + a_2 \cos(x),$$

$$\hat{y} = a_0 + a_1T + a_2P + a_3TP.$$

All can be fitted using least squares after constructing suitable features.

# Model Complexity and Polynomial Degree

The polynomial degree is a modelling choice.

| Degree    | Model flexibility | Risk         |
|-----------|-------------------|--------------|
| 1         | Low               | Underfitting |
| 2–4       | Moderate          | Often useful |
| Very high | High              | Overfitting  |

## Key idea

A more flexible model may fit the training data better, but that does not guarantee better predictions on new data.

## Optional Extension: Regularised Least Squares

If a model is too flexible or the data matrix is ill-conditioned, we can penalise large coefficients.

$$J(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_2^2.$$

The solution is

$$\hat{\mathbf{w}} = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$$

- $\lambda = 0$ : ordinary least squares.
- Larger  $\lambda$ : stronger penalty on large coefficients.

# Training, Validation, and Test Sets

We usually split the data into three parts.

| Dataset part   | Purpose                                                      |
|----------------|--------------------------------------------------------------|
| Training set   | Used to fit the model parameters.                            |
| Validation set | Used to compare models and tune hyperparameters.             |
| Test set       | Used only at the end to estimate performance on unseen data. |

## Warning

Repeatedly tuning a model based on test-set performance makes the test set no longer independent.

# Generalisation

A model generalises well if it performs well on new data drawn from the same underlying problem.

| Case         | Description                                                                    |
|--------------|--------------------------------------------------------------------------------|
| Underfitting | Model too simple; poor training and test performance.                          |
| Good fit     | Captures the main pattern; good test performance.                              |
| Overfitting  | Model follows noise; excellent training performance but poor test performance. |

The objective is not to minimise training error at all costs. The objective is useful prediction on unseen data.

# Regression Metrics

For regression, common error metrics include:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|,$$

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2,$$

$$\text{RMSE} = \sqrt{\text{MSE}}.$$

- MAE is easy to interpret and less sensitive to large errors.
- MSE penalises large errors more strongly.
- RMSE has the same units as the target variable.



# The Coefficient of Determination: $R^2$

The coefficient of determination compares the model to a simple baseline that always predicts the mean.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}.$$

- $R^2 = 1$ : perfect predictions.
- $R^2 = 0$ : no better than predicting the mean.
- $R^2 < 0$ : worse than predicting the mean.

## Caution

A high  $R^2$  on the training data does not necessarily imply good generalisation.

# Classification: Scores, Probabilities, and Classes

In binary classification, the target is often

$$y_i \in \{0, 1\}.$$

Many classifiers first compute a score or probability:

$$p_i = P(y_i = 1 \mid \mathbf{x}_i).$$

Then a decision threshold is applied:

$$\hat{y}_i = \begin{cases} 1, & p_i \geq 0.5, \\ 0, & p_i < 0.5. \end{cases}$$

The threshold can be changed when false positives and false negatives have different costs.

# Logistic Regression

Logistic regression is a supervised learning model for classification.

First compute a linear score:

$$z_i = \mathbf{w}^\top \mathbf{x}_i + b.$$

Then convert the score into a probability using the sigmoid function:

$$p_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}.$$

## Interpretation

$p_i$  is the model's estimated probability that example  $i$  belongs to class 1.

# Why Not Use Linear Regression for Classification?

Linear regression predicts

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b.$$

For classification, this can be problematic:

- predictions can be below 0 or above 1;
- squared error is not naturally matched to binary labels;
- we often want calibrated probabilities, not just numerical scores.

Logistic regression instead uses

$$0 < \sigma(z) < 1,$$

so the model output can be interpreted as a probability.

# Logistic Regression Loss Function

For binary labels  $y_i \in \{0, 1\}$ , logistic regression uses the negative log-likelihood:

$$J(\mathbf{w}, b) = - \sum_{i=1}^n [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] .$$

where

$$p_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b).$$

- If  $y_i = 1$ , the loss is small when  $p_i$  is close to 1.
- If  $y_i = 0$ , the loss is small when  $p_i$  is close to 0.

# Logistic Regression as an Optimisation Problem

Logistic regression estimates its parameters by solving

$$(\mathbf{w}^*, b^*) = \arg \min_{\mathbf{w}, b} J(\mathbf{w}, b).$$

Unlike ordinary least squares, there is generally no closed-form solution like

$$(X^\top X)^{-1} X^\top \mathbf{y}.$$

Instead, we use iterative numerical optimisation.

## Important contrast

Linear regression has a closed-form least-squares solution. Logistic regression is usually fitted by an optimisation algorithm.

# Logistic Regression: Gradient Form

Using augmented feature vectors

$$\tilde{\mathbf{x}}_i = \begin{bmatrix} 1 \\ \mathbf{x}_i \end{bmatrix},$$

the predicted probability for observation  $i$  is

$$p_i = \sigma\left(\boldsymbol{\theta}^\top \tilde{\mathbf{x}}_i\right).$$

Collecting all predictions,

$$\mathbf{p} = [p_1 \quad p_2 \quad \cdots \quad p_n]^\top.$$

The gradient of the average logistic loss is

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \frac{1}{n} \tilde{X}^\top (\mathbf{p} - \mathbf{y}).$$

Therefore, gradient descent gives

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \frac{\alpha}{n} \tilde{X}^\top (\mathbf{p}^{(k)} - \mathbf{y})$$

# Decision Boundaries

In logistic regression, the class prediction changes when

$$p_i = 0.5.$$

Since

$$\sigma(z_i) = 0.5 \iff z_i = 0,$$

we get the decision boundary

$$\mathbf{w}^\top \mathbf{x} + b = 0.$$

- In two dimensions, this is a line.
- In three dimensions, this is a plane.
- In higher dimensions, this is a hyperplane.



# Confusion Matrix

For binary classification, predictions can be summarised as:

|                          | <b>Predicted positive</b> | <b>Predicted negative</b> |
|--------------------------|---------------------------|---------------------------|
| <b>Actually positive</b> | True positive             | False negative            |
| <b>Actually negative</b> | False positive            | True negative             |

- A false positive is a false alarm.
- A false negative is a missed detection.
- Which one is worse depends on the application.

# Classification Metrics

From the confusion matrix, we can compute:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN},$$

$$\text{Precision} = \frac{TP}{TP + FP},$$

$$\text{Recall} = \frac{TP}{TP + FN},$$

$$F_1 = 2 \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}}.$$

Accuracy can be misleading when one class is much more common than the other.

# Threshold Choice Matters

The default threshold for binary classification is often 0.5:

$$\hat{y} = 1 \quad \text{if} \quad p \geq 0.5.$$

But this threshold is a decision choice, not a law of nature.

|                         |                                                                            |
|-------------------------|----------------------------------------------------------------------------|
| <b>Lower threshold</b>  | More positives predicted; usually higher recall, more false positives.     |
| <b>Higher threshold</b> | Fewer positives predicted; usually higher precision, more false negatives. |

In safety-critical fault detection, a missed fault may be more costly than a false alarm.

# Receiver Operating Characteristic (ROC) Curve

Changing the classification threshold changes the balance between **detected positives** and **false alarms**.

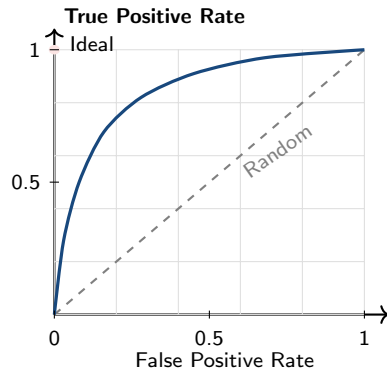
$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}.$$

The ROC curve plots

TPR against FPR

over all possible thresholds.

- Upper-left: ideal performance
- Diagonal: random guessing
- Larger AUC: better discrimination



AUC = area under the ROC curve

0.5 : random,      1 : perfect.

# Data Preparation

Before training a supervised learning model, we often need to handle:

- missing values;
- categorical variables;
- different feature scales;
- outliers;
- duplicated or inconsistent records;
- imbalanced classes;
- data leakage.

## Data leakage

Data leakage occurs when information is used during training that would not be available at prediction time.

# Feature Scaling

Some models are sensitive to the numerical scale of features.

For example,  $k$ -nearest neighbours uses distances:

$$\|\mathbf{x}_i - \mathbf{x}_j\|_2.$$

If one feature is measured in thousands and another is measured between 0 and 1, the large-scale feature may dominate the distance.

A common standardisation is

$$x_{ij}^{\text{scaled}} = \frac{x_{ij} - \mu_j}{s_j},$$

where  $\mu_j$  and  $s_j$  are computed from the training data.

# Model Selection

Different models and hyperparameters can be compared using validation data.

| Choice          | Examples                                              |
|-----------------|-------------------------------------------------------|
| Model class     | linear regression, tree, logistic regression, $k$ -NN |
| Hyperparameters | polynomial degree, tree depth, number of neighbours   |
| Metric          | MAE, RMSE, accuracy, precision, recall                |

## Rule

Use the validation set for model selection. Use the test set only for final assessment.

# Use of Supervised Learning with Care!

A supervised learning model can be wrong for many reasons.

- The training data may be biased or incomplete.
- The future may differ from the past.
- The selected features may miss important information.
- The model may overfit noise.
- Different types of error may have very different consequences.

## Practical message

Always ask: what happens if the model is wrong?



# End-to-End Worked Example Structure

A complete supervised learning example can follow this structure.

- 1 Define the prediction task.
- 2 Identify features and target.
- 3 Inspect and prepare the data.
- 4 Split into training, validation, and test sets.
- 5 Train a simple baseline model.
- 6 Evaluate on validation data.
- 7 Try a more flexible model.
- 8 Compare metrics and inspect errors.
- 9 Evaluate once on the test set.
- 10 Decide whether the model is useful for the intended purpose.

- Supervised learning learns from labelled examples.
- Regression predicts numerical values; classification predicts categories.
- Training usually means minimising a loss function.
- Linear regression can be solved by least squares.
- Polynomial regression is linear in the coefficients after feature transformation.
- Logistic regression is fitted by iterative numerical optimisation.
- Generalisation matters more than training error.
- Evaluation metrics must match the purpose of the model.